

Merge Sort

To understand merge sort, we take an unsorted array as the following –



We know that merge sort first divides the whole array iteratively into equal halves unless the atomic values are achieved. We see here that an array of 8 items is divided into two arrays of size 4.



This does not change the sequence of appearance of items in the original. Now we divide these two arrays into halves.



We further divide these arrays and we achieve atomic value which can no more be divided.



Now, we combine them in exactly the same manner as they were broken down. Please note the color codes given to these lists.

We first compare the element for each list and then combine them into another list in a sorted manner. We see that 14 and 33 are in sorted positions. We compare 27 and 10 and in the target list of 2 values we put 10 first, followed by 27. We change the order of 19 and 35 whereas 42 and 44 are placed sequentially.



In the next iteration of the combining phase, we compare lists of two data values, and merge them into a list of found data values placing all in a sorted order.



After the final merging, the list should look like this –



```

#include<iostream>
using namespace std;

void Merge(int left[], int right[], int arr[], int nL, int nR) {

    int i = 0;
    int j = 0;
    int k = 0;

    while (i < nL && j < nR) {
        if (left[i] < right[j]) {
            arr[k] = left[i];
            i = i + 1;
        } else {
            arr[k] = right[j];
            j = j + 1;
        }
        k = k + 1;
    }

    while (i < nL) {
        arr[k] = left[i];
        i = i + 1;
        k = k + 1;
    }

    while (j < nR) {
        arr[k] = right[j];
        j = j + 1;
        k = k + 1;
    }
}

void MergeSort(int arr[], int n) {
    if (n < 2) {
        return;
    }

    int mid = n / 2;
    int left[mid];
    int right[n - mid];

    for (int i = 0; i < mid; i++) {
        left[i] = arr[i];
    }
}

```

```

for (int i = mid; i < n; i++) {
    right[i - mid] = arr[i];
}

MergeSort(left, mid);
MergeSort(right, n - mid);
Merge(left, right, arr, mid, n - mid);
}

void PrintArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
}

int main() {
    int arr[] = { 6, 2, 3, 1, 9, 10, 15, 13, 12, 17 };
    int n = sizeof(arr) / sizeof(arr[0]);
    MergeSort(arr, n);
    PrintArray(arr, n);
    return 0;
}

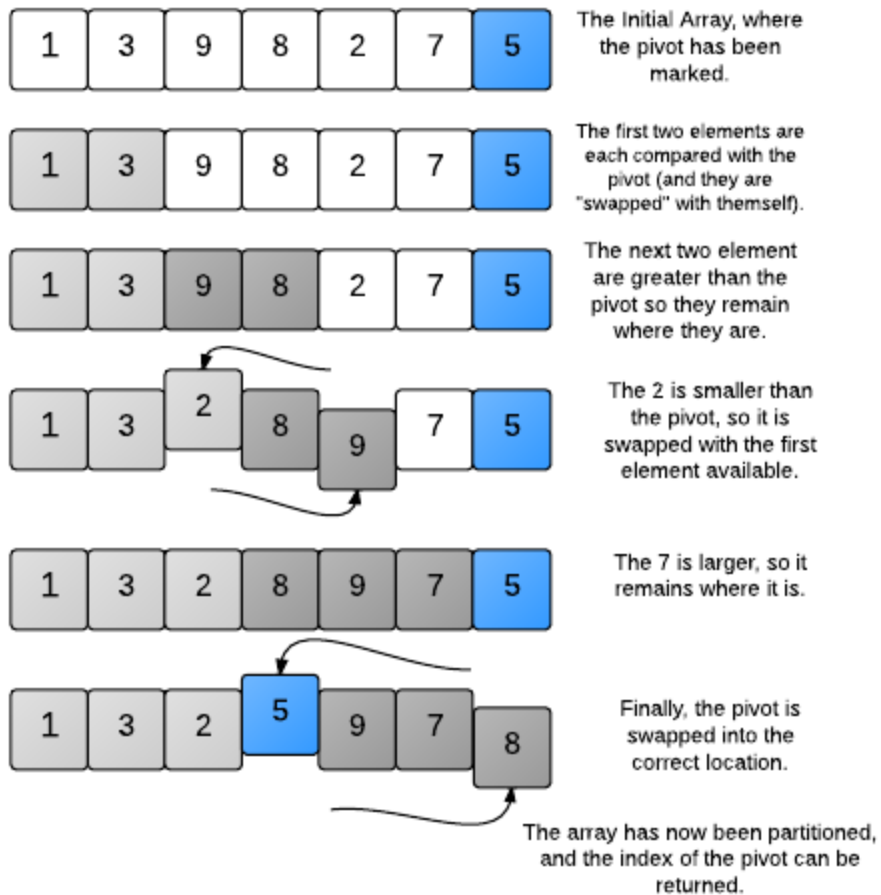
```

Quick Sort

Quick sort is a highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays. A large array is partitioned into two arrays one of which holds values smaller than the specified value, say pivot, based on which the partition is made and another array holds values greater than the pivot value.

Quicksort partitions an array and then calls itself recursively twice to sort the two resulting subarrays.

Partitioning an array



```
#include <iostream>
using namespace std;
```

```
int partition(int *A, int start, int end)
{
    int pivot = A[end];
    int partitionIndex = start;

    for (int i=start ; i<end ; i++)
    {
        if (A[i] <= pivot)
        {
            swap (A[i] , A[partitionIndex]);
            partitionIndex++;
        }
    }
}
```

```
    }  
    swap(A[partitionIndex], A[end]);  
    return partitionIndex;  
}
```

```
void Quicksort (int *A, int start, int end)  
{  
    if(start < end){  
        int partitionIndex=partition(A ,start,end);  
        Quicksort(A,start,partitionIndex-1);  
        Quicksort(A,partitionIndex+1,end);  
    }  
}
```

```
int main()  
  
{  
    int A[ ]={11,55,10,34,18,12,19,50};  
  
    Quicksort (A,0,7);  
  
    for(int i=0; i < 8; i++)  
    {  
        cout<<A[i]<<" ";  
    }  
  
}
```